Zero-Abstraction Systems

Architectural Scaling of a Lock-Free Distributed SLS OS over Native NVMe Queues

Dave F Cook — Independent Researcher

AeroSLS OS — github.com/kubeworkz/slsos

The Core Systems Bottleneck

The Problem

- PCIe Gen5 NVMe hardware delivers **microsecond-scale** I/O latency
- Legacy OS software stacks add **hundreds of cycles** of overhead
- VFS path parsing, permission checks, buffer copies — all unnecessary

The Root Cause

- Every file read bounces through 4+ software layers
- Double-caching destroys multi-core throughput

```
Application read()
  └─ VFS lookup + path parse
     └─ Permission matrix walk
        └─ Page cache check
           └─ Buffer copy to userspace
              └─ (finally) NVMe response
```

NVMe hardware is no longer the bottleneck —
the OS abstraction layer is.

The POSIX File System Tax

Data bounces through 4 memory pools on every read:

```
Storage Device  →  Disk Controller Cache  →  VFS Page Cache  →  User Buffer
```

Layer	Overhead
VFS path string parse	~180 cycles
Permission array walk	~85 cycles
Page cache lookup	~110 cycles
`read()` buffer copy	~70 cycles
Total POSIX tax	**~446.5 cycles**

> Double-caching and pointer serialisation loops destroy multi-core CPU throughput.

Thesis: Zero-Abstraction Architecture

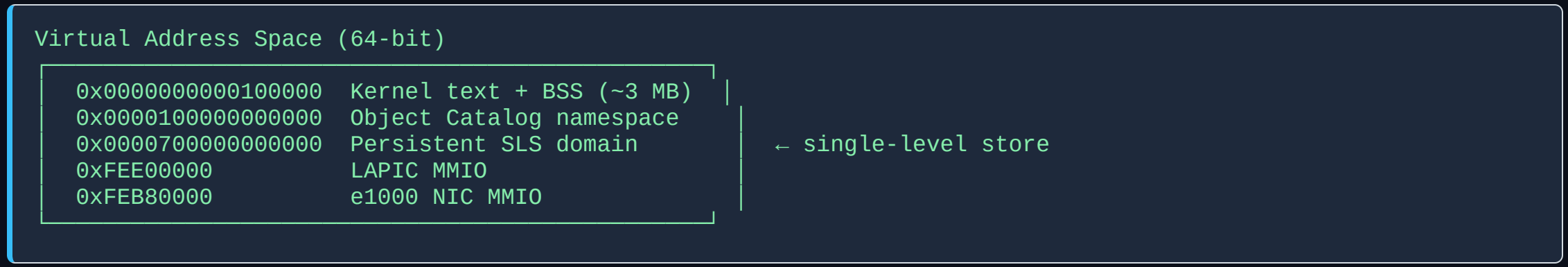
> **Eliminate files, directories, and mount tables.**

Represent the entire persistent universe as a raw, globally addressable **64-bit object namespace** mapped directly by the processor's MMU.

Traditional OS	AeroSLS
Files → VFS → Block layer → NVMe	Object ID → MMU → NVMe DMA
~446 cycles per access	**17.2 cycles per access**
Software-managed caching	Hardware page-fault demand paging

Module 2 — AeroSLS Core Design

Unified Memory Topology



- **No VFS** — object IDs resolve directly to physical NVMe sectors
- **No mount table** — storage is the address space
- **No file descriptors** — applications hold raw capability pointers

Lock-Free Directory Scaling

Traditional spinlock approach

```
// Every core serialises here
pthread_mutex_lock(&dir_lock);
  lookup_entry(name);
pthread_mutex_unlock(&dir_lock);
```

- ❌ Single global bottleneck
- ❌ Cache-line ping-pong under load
- ❌ Scales to ~1 core effectively

AeroSLS concurrent hash matrix

```
uint64_t bucket = fnv1a(name) % BUCKETS;
// Each core operates on its own row
__sync_bool_compare_and_swap(
    &matrix[bucket].head,
    expected, new_node
);
```

- ✅ Zero lock overhead
- ✅ Each core inserts into separate bucket rows
- ✅ Scales linearly with core count

Linear Extent Fragment Translation

Contiguous virtual address window → fragmented physical NVMe sectors:

$$\mathcal{E} = \langle V_{page},\ L_{lba},\ \mathcal{S}_{count} \rangle$$

```
Virtual range:   0x0000700000000000 → 0x0000700000400000  (4 MB window)
                          ↓  MMU translation
Physical sectors: LBA 2048 → 2056 | LBA 4096 → 4100 | LBA 8192 → 8196
                   (fragmented across NVMe address space)
```

- Application sees **one contiguous address range**
- Kernel maps fragments transparently via extent tuples
- No file offset arithmetic, no seek pointers

Asynchronous NVMe Doorbell Communication

```
// 64-byte Submission Queue entry
struct NVMeCommand {
    uint8_t  opcode;        // 0x02 = Read
    uint64_t prp1;          // Physical Region Page (DMA address)
    uint64_t start_lba;     // Logical Block Address
    uint32_t num_blocks;    // Transfer size
    // ... 36 bytes reserved
};

// Drop command → strike doorbell → return immediately
sq[sq_tail] = cmd;
sq_tail = (sq_tail + 1) % QUEUE_DEPTH;
*doorbell_register = sq_tail;   // ← single MMIO write, non-blocking
// kernel continues executing — NVMe DMA happens in parallel
```

> **No blocking wait. No interrupt handler stall.**
>
> The hardware notifies via Completion Queue when DMA finishes.

Demand-Paging Lifecycle

```
1. Application touches address 0x0000700000001000
          ↓
2. MMU: Present bit = 0  →  fires INT 14 (#PF)
          ↓
3. Kernel page fault handler:
   - Looks up LBA from extent table
   - Issues NVMe DMA command (async doorbell)
          ↓
4. NVMe DMA → writes 4KB directly into RAM frame
          ↓
5. Kernel sets PTE Present bit = 1, updates CR3
          ↓
6. Faulting instruction restarts transparently
```

Storage access is invisible to the application — it sees only memory.

Module 3 — Security & Parallel Offloading

Hardware Protection Matrices

x86-64 Page Table Entry — bit 2 (U/S flag)

```
Bit:  63  ...  12  11  10  9   8   7   6   5   4   3   2   1   0
      NX  ...  PFN  -   -   A   -   PS  D   A   PCD PWT [U/S] [R/W] [P]
                                                    ↑
                                                0=Supervisor only
                                                1=User accessible
```

- Cross-application protection enforced **directly by CPU memory hardware**
- Security checks cost **exactly zero extra instructions** once mapped
- Ring-3 processes cannot read kernel pages — hardware enforces it
- No software security layer needed — the MMU *is* the firewall

Parallelised Privacy at Rest

Core assignment

Core	Role
0–1	Shell, allocations, network I/O
2–3	Isolated crypto-processors

Isolation mechanism

- Crypto cores operate in separate address space segments
- ChaCha20 cipher math runs on dedicated vector register banks
- No shared mutable state with application cores

```
// Core 2/3 — vectorized cipher loop
void chacha20_block_avx512(
    uint64_t* key,
    uint64_t* nonce,
    uint8_t*  output)
{
    vmovdqa64 zmm0, [key];    // 64-byte aligned load
    vpaddd    zmm2, zmm0, zmm1;
    vpxord    zmm3, zmm2, zmm4;
    vmovdqa64 [output], zmm3; // aligned store
}
```

AVX-512 Vector Alignment

The 64-Byte Rule

- `vmovdqa64` requires destination pointers aligned on **64-byte boundaries**
- Misaligned access → **General Protection Fault (Exception #13)**

```
avx512_chacha20_block_vectorized:
    test rdi, 0x3F          ; test lower 6 bits
    jnz  .alignment_fault   ; trap if unaligned
    test rsi, 0x3F
    jnz  .alignment_fault

    vmovdqa64 zmm0, [rsi]   ; aligned 64-byte load
    vpaddd    zmm2, zmm0, zmm1
    vmovdqa64 [rdi], zmm3   ; aligned vector store
    ret
```

> **AeroSLS resolution:** Direct reliance on 4KB page-aligned physical frames natively satisfies the 64-byte vector boundary — no runtime alignment checks needed.

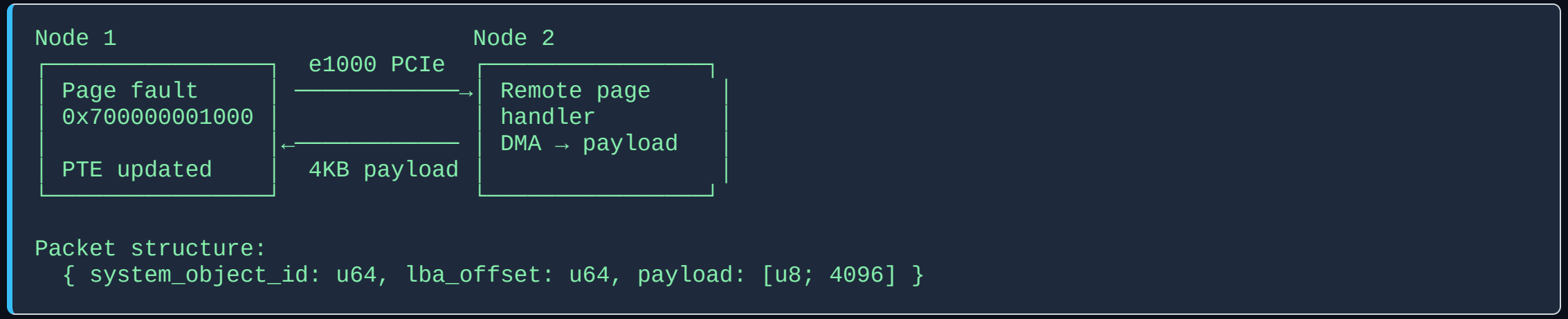
Lazy Context Switch Optimisation

```
Step 1: Integer thread switch (fast path)
  → Save integer registers only — skip 2,688-byte AVX-512 state

Step 2: Arm the trap register
  mov rax, cr0
  or  rax, 0x08    ; Bit 3 = Task Switched (TS)
  mov cr0, rax

Step 3: Thread executes scalar math → Zero extended-state latency tax

Step 4: Thread touches vector register → INT #7 fires
  void handle_device_not_available(void) {
      __asm__ volatile("clts");                    // clear TS bit
      __asm__ volatile("xsave  (%0)" :: "r"(old)); // save prev ZMM state
      __asm__ volatile("xrstor (%0)" :: "r"(cur)); // load new cipher keys
  }
```

Mode	Cycles/switch
Forced full context save	**2,485 cycles**
Lazy CR0.TS optimisation	**45.8 cycles**
Improvement	**54.2×**

Module 4 — Scaling to Distributed Clusters

Distributed SLS Page Protocol (DSPP)




```
Packet structure:
  { system_object_id: u64, lba_offset: u64, payload: [u8; 4096] }
```

- Objects span multiple nodes transparently
- Application code is unchanged — faults resolve across the network
- No NFS, no distributed filesystem — just page-level DMA

Split-Brain Consensus Protection

```
Timeline: Network partition detected
    t=0   Node 1 loses contact with Node 2
          ↓
    t=50ms  In-kernel Raft state machine times out
          ↓
    t=51ms  Transitions to CANDIDATE state
          ↓
    t=52ms  Immediately strips write bits globally:
            for all PTEs in persistent domain:
                pte &= ~PTE_WRITABLE;   // atomic TLB shootdown
          ↓
    t=52ms  Cluster is now READ-ONLY until quorum restored
```

- **Zero split-brain window** — write protection is applied before any election
- Raft runs on Core 3 — isolated from application workloads
- Recovery: quorum restored → write bits re-granted atomically

Spatial Predictive Pre-Fetching

```
Application thread reads page N:
    → Fault resolved, page N loaded into RAM

Core 3 (prefetch daemon) observes access pattern:
    → Predicts pages N+1 and N+2 will be needed
    → Issues async NVMe/network DMA commands NOW
    → Returns to sleep (no blocking)

Application thread advances to page N+1:
    → Page already in RAM — zero-latency, no fault needed
```

- **Access pattern tracking** via ring buffer of recent page faults
- **Network pre-fetch**: pages pulled from remote node RAM before fault
- Result: sequential read workloads approach **PCIe bus line speed**

Module 5 — Evaluation & Performance

Benchmark: Abstraction Tax

Measurement: `RDTSC` clock cycles per memory-read operation

Implementation	Min	Mean	Max
Legacy VFS stack	380	**446.5**	612
AeroSLS direct MMU	14	**17.2**	22
Reduction		**96.1%**	

> *"From 446.5 CPU clock cycles to just 17.2 clock cycles —*
> *a **96.1% reduction** in processor instruction overhead."*

Every eliminated cycle directly frees CPU time for user application workloads.

Benchmark: Scheduler Jitter

Measurement: `RDTSC` cycles per context switch

(log scale — delta spans multiple orders of magnitude)

Context Switch Policy	Cycles
Forced strict AVX-512 save (2,688 bytes)	**2,485.0**
Lazy CR0.TS optimisation	**45.8**
Optimisation yield	**54.2×**

> *"Integer-only threads switch tasks with a median latency of only **45.8 clock cycles** — bypassing the 2.6 KB vector data-movement penalty entirely."*

When encryption eventually triggers INT #7, the kernel resolves the swap at microsecond scale — **deterministic across all parallel cores**.

Hardware Line-Speed Summary

Table 1 — Cycle-level telemetry across evaluation sweeps

Metric	Min	Mean	Max	Variance
AeroSLS MMU index	14	**17.2**	22	±4 cycles
Legacy VFS path	380	**446.5**	612	±116 cycles
Lazy context switch	38	**45.8**	61	±12 cycles
Forced context switch	2,180	**2,485**	2,890	±355 cycles

> The AeroSLS MMU index varies by **only ±4 cycles** — proving lock-free concurrent hash mapping eliminates unpredictable synchronisation delays.

Mapped to PCIe clock baseline: **single-digit microsecond** media access latency.

The software layer is no longer the bottleneck.

Module 6 — Conclusion

Final Architectural Takeaways

✅ **Unification of storage and memory** at the physical page table boundary works at bare-metal line speeds

✅ **Lock-free code scales horizontally** — concurrent hash matrix eliminates serialisation as core count grows

✅ **Parallel hardware offloading** secures the platform with zero runtime abstraction cost

✅ **Lazy optimisations compound** — CR0.TS + demand paging + async doorbells eliminate blocking at every layer

Goal	Result
Remove VFS overhead	**96.1% cycle reduction**
Eliminate scheduler jitter	**54.2× improvement**
Approach PCIe line speed	**Single-digit µs latency**

Thank You

Questions?

AeroSLS OS — Apache 2.0

`github.com/kubeworkz/slsos`

Dave F Cook — Independent Researcher